QuillAudits

AUDIT REPORT

February 2026

For

Zynk

Table of Content

Executive Summary 04

Number of Security Issues per Severity 05

Summary of Issues 06

Checked Vulnerabilities 08

Techniques and Methods 10

Types of Severity 11

Types of Issues 12

Severity Matrix 13

Medium Severity Issues 14

1. Protocol Initialization is Vulnerable to Front-Running 14

2. Duplicate Mutable Accounts Enable Phantom Orders 15

3. Delimiter Injection in attest_order Signature Messages 17

4. Zero-Amount Transfer in pull_and_create_order 18

5. Incomplete Ed25519 Signature Verification — Missing num_signatures and Offset Validation 19

Low Severity Issues 23

6. Arithmetic Operations Should Use Checked Math 23

7. No Duplicate Check in Whitelisted Token Mints 24

8. No Support for Token-2022 25

9. Attest_order signature does not include amount 25

10. Custom close_account Does Not Zero Discriminator — Account Revival Attack 26

Informational Issues 28

11. Monolithic Code Structure 28

12. execute_consensus Double-Close on Timelock Account 29

13. Manual Struct Size Calculation 30

Closing Summary & Disclaimer 31

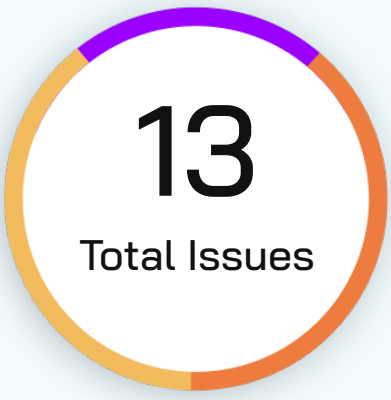
Executive Summary

Project Name	Zynk Labs
Project URL	https://www.zynk.money/
Overview	The Zynk Protocol is a Solana-based smart contract that facilitates token transfers between different parties through a managed operator system. It implements a sophisticated architecture with timelock mechanisms, signature verification, and order tracking capabilities.
Audit Scope	The scope of this Audit was to analyze the Zynk Labs Smart Contracts for quality, security, and correctness.
Source Code link	https://github.com/Zynklabs/zynk-protocol-solana-programs/blob/zynk-core-v3/programs/zynk-core/src/lib.rs
Branch	zynk-core-v3
Contracts in Scope	lib.rs
Commit Hash	c152cb7fbecd11f2aed45908f81b966d978f08c0
Language	Rust
Blockchain	Solana
Method	Manual Analysis, Functional Testing, Automated Testing
Review 1	4th February 2026 - 11th February 2026
Updated Code Received	14th Feb 2026 - 18th February 2026
Review 2	16th Feb 2026 - 18th February 2026
Fixed In	5bb6d021cca703679b12fffd94f9f88581c21686

Verify the Authenticity of Report on QuillAudits Leaderboard:

https://www.quillaudits.com/leaderboard

Number of Issues per Severity



Critical	0 (0.0%)
High	0 (0.0%)
Medium	5 (38.4%)
Low	5 (38.4%)
Informational	3 (23.2%)

Severity

Issues	Critical	High	Medium	Low	Informational
Open	0	0	0	0	0
Acknowledged	0	0	0	0	**1**
Partially Resolved	0	0	0	0	0
Resolved	0	0	**5**	**5**	**2**

Summary of Issues

Issue No.	Issue Title	Severity	Status
1	Protocol Initialization is Vulnerable to Front-Running	Medium	Resolved
2	Duplicate Mutable Accounts Enable Phantom Orders	Medium	Resolved
3	Delimiter Injection in attest_order Signature Messages	Medium	Resolved
4	Zero-Amount Transfer in pull_and_create_order	Medium	Resolved
5	Incomplete Ed25519 Signature Verification — Missing num_signatures and Offset Validation	Medium	Resolved
6	Arithmetic Operations Should Use Checked Math	Low	Resolved
7	No Duplicate Check in Whitelisted Token Mints	Low	Resolved
8	No Support for Token-2022	Low	Resolved
9	Attest_order signature does not include amount	Low	Resolved

Issue No.	Issue Title	Severity	Status
10	Custom close_account Does Not Zero Discriminator — Account Revival Attack	Low	Resolved
11	Monolithic Code Structure	Informational	Acknowledged
12	execute_consensus Double-Close on Timelock Account	Informational	Resolved
13	Manual Struct Size Calculation	Informational	Resolved

Checked Vulnerabilities

We have scanned the solana program for commonly known and more specific vulnerabilities. Here are some of the commonly known vulnerabilities that we considered:

- Signer authorization
- Account data matching
- Sysvar address checking
- Owner checks
- Type cosplay
- Initialization
- Arbitrary cpi
- Duplicate mutable accounts
- Bump seed canonicalization
- PDA Sharing
- Incorrect closing accounts
- Missing rent exemption checks
- Arithmetic overflows/underflows
- Numerical precision errors
- Solana account confusions
- Casting truncation
- Insufficient SPL token account verification
- Signed invocation of unverified programs

Techniques and Methods

Throughout the audit of Solana Programs, care was taken to ensure:

- The overall quality of code
- Use of best practices
- Code documentation and comments, match logic and expected behavior
- Token distribution and calculations are as per the intended behavior mentioned in the whitepaper
- Efficient use of gas
- Code is safe from re-entrancy and other vulnerabilities

The following techniques, methods, and tools were used to review all the smart contracts:

Structural Analysis

In this step, we have analyzed the design patterns and structure of smart contracts. A thorough check was done to ensure the smart contract is structured in a way that will not result in future problems.

Static Analysis

A static Analysis of Smart Contracts was done to identify contract vulnerabilities. In this step, a series of automated tools are used to test the security of smart contracts.

Code Review / Manual Analysis

Manual Analysis or review of code was done to identify new vulnerabilities or verify the vulnerabilities found during the static analysis. Contracts were completely manually analyzed, their logic was checked and compared with the one described in the whitepaper. Besides, the results of the automated analysis were manually verified.

Gas Consumption

In this step, we have checked the behavior of smart contracts in production. Checks were done to know how much gas gets consumed and the possibilities of optimization of code to reduce gas consumption.

Types of Severity

Every issue in this report has been assigned to a severity level. There are five levels of severity, and each of them has been explained below.

Critical: Immediate and Catastrophic Impact

Critical issues are the ones that an attacker could exploit with relative ease, potentially leading to an immediate and complete loss of user funds, a total takeover of the protocol's functionality, or other catastrophic failures. Critical vulnerabilities are non-negotiable; they absolutely must be fixed.

High (H): Significant Risk of Major Loss or Compromise

High-severity issues represent serious weaknesses that could result in significant financial losses for users, major malfunctions within the protocol, or substantial compromise of its intended operations. While exploiting these vulnerabilities might require specific conditions to be met or a moderate level of technical skill, the potential damage is considerable. These findings are critical and should be addressed and resolved thoroughly before the contract is put into the Mainnet.

Medium (M): Potential for Moderate Harm Under Specific Circumstances

Medium-severity bugs are loopholes in the protocol that could lead to moderate financial losses or partial disruptions of the protocol's intended behavior. However, exploiting these vulnerabilities typically requires more specific and less common conditions to occur, and the overall impact is generally lower compared to high or critical issues. While not as immediately threatening, it's still highly recommended to address these findings to enhance the contract's robustness and prevent potential problems down the line.

Low (L): Minor Imperfections with Limited Repercussions

Low-severity issues are essentially minor imperfections in the smart contract that have a limited impact on user funds or the core functionality of the protocol. Exploiting these would usually require very specific and unlikely scenarios and would yield minimal gain for an attacker. While these findings don't pose an immediate threat, addressing them when feasible can contribute to a more polished and well-maintained codebase.

Informational (I): Opportunities for Improvement, Not Immediate Risks

Informational findings aren't security vulnerabilities in the traditional sense. Instead, they highlight areas related to the clarity and efficiency of the code, gas optimization, the quality of documentation, or adherence to best development practices. These findings don't represent any immediate risk to the security or functionality of the contract but offer valuable insights for improving its overall quality and maintainability. Addressing these is optional but often beneficial for long-term health and clarity.

Types of Issues

Open

Security vulnerabilities identified that must be resolved and are currently unresolved.

Resolved

These are the issues identified in the initial audit and have been successfully fixed.

Acknowledged

Vulnerabilities which have been acknowledged but are yet to be resolved.

Partially Resolved

Considerable efforts have been invested to reduce the risk/impact of the security issue, but are not completely resolved.

Severity Matrix

Impact

Likelihood	High	Medium	Low
High	Critical	High	Medium
Medium	High	Medium	Low
Low	Medium	Low	Low

Impact

- **High** - leads to a significant material loss of assets in the protocol or significantly harms a group of users.
- **Medium** - only a small amount of funds can be lost (such as leakage of value) or a core functionality of the protocol is affected.
- **Low** - can lead to any kind of unexpected behavior with some of the protocol's functionalities that's not so critical.

Likelihood

- High - attack path is possible with reasonable assumptions that mimic on-chain conditions, and the cost of the attack is relatively low compared to the amount of funds that can be stolen or lost.
- Medium - only a conditionally incentivized attack vector, but still relatively likely.
- Low - has too many or too unlikely assumptions or requires a significant stake by the attacker with little or no incentive.

Medium Severity Issues

Protocol Initialization is Vulnerable to Front-Running

Resolved

Path

lib.rs

Description

The initialize function doesn't check who's calling it - whoever calls it first becomes the Admin:

```
// initialize (lib.rs:259-273)
pub fn initialize(
    ctx: C><Initialize>,
    zynk_op_wallet: Pubkey,
    guardian: Pubkey,
    manager: Pubkey,
) -> Result<()> {
    let c>= &mut ctx.accounts.c>;
    c>.admin = ctx.accounts.admin.key(); // Caller becomes admin
    // …
}
```

Since the Config PDA uses constant seeds (b"config"), anyone can calculate its address ahead of time. An attacker monitoring for deployment transactions could front-run the legitimate initialization call.

Impact

- An attacker can become the Admin by front-running the legitimate deployment
- Full control over protocol configuration, including all privileged operations
- Complete protocol takeover before the legitimate team can react
- Requires redeploying the program to recover

Recommendation

Restrict initialize to the program's upgrade authority, put a check in the initialize function that only a specific address i.e ADMIN , can initialize the program.

Fixed in Commit hash

e32e0d1856d1be0f7080caf848379b9a4239d4bf

Zynk Team's Comment

Currently, we perform the initialization as soon as our program is deployed. Given that, for every deployment we generate a new programId, an attacker must constantly keep track of our deployment, in order to derive the config PDAs address and hit the `initialize()` method before our script does.

Even in that case, if our `initialize()` fails or the config is not updated as per our requirements, we can always ditch/retire that program.
This is a risk we are willing to take, as we do not see any incentive for the attacker to use the invalid program..

Duplicate Mutable Accounts Enable Phantom Orders

Resolved

Path

lib.rs

Description

No constraint prevents zow_token_account == beneficiary_token_account in the CreateOrder account struct. When both accounts point to the same SPL token account, the token transfer becomes a self-transfer (no-op in SPL Token), but the OrderTracker still records the full amount_out as a debt obligation. This creates phantom orders with recorded debt but zero actual token movement.

```
// lib.rs:1082-1094 - No constraint preventing account overlap
#[account(mut,
    constraint = zow_token_account.owner == config.zynk_op_wallet @
CustomError::UnauthorizedSigner,
    constraint =
config.whitelisted_token_mints.contains(&zow_token_account.mint) @
CustomError::InvalidTokenMint
)]
pub zow_token_account: Box<Account<'info, TokenAccount>>,
#[account(mut,
    constraint = beneficiary_token_account.mint == zow_token_account.mint @
CustomError::InvalidTokenMint,
    // MISSING: constraint = beneficiary_token_account.key() !=
zow_token_account.key()
)]
pub beneficiary_token_account: Box<Account<'info, TokenAccount>>,
```

Remediation Suggestion

```
#[account(mut,
    constraint = beneficiary_token_account.mint == zow_token_account.mint @
CustomError::InvalidTokenMint,
    constraint = beneficiary_token_account.key() != zow_token_account.key() @
CustomError::DuplicateAccounts,
)]
pub beneficiary_token_account: Box<Account<'info, TokenAccount>>,
```

POC

```
it("PoC-5: Phantom order via self-transfer (F-07)", async () => {
    const orderId = generateOrderId();
    const zynkOpWalletOwner = zynkOpWallet.publicKey;
    const [orderTrackerPDA] = PublicKey.findProgramAddressSync(
      [Buffer.from("order_tracker"), zynkOpWalletOwner.toBuffer(), orderId],
      program.programId
    );

    const amount = new anchor.BN(500_000_000);
    const zowBalanceBefore = await getAccount(provider.connection,
zynkOpTokenAccount);
```

```
    await program.methods
      .createOrder(Array.from(partnerId), Array.from(orderId), amount, null,
null)
      .accounts({
        config: configPDA,
        manager: manager.publicKey,
        partnerDepositVault: partnerDepositVaultPDA,
        pdvTokenAccount: null,
        zynkOpWallet: zynkOpWallet.publicKey,
        zowTokenAccount: zynkOpTokenAccount,
        beneficiaryTokenAccount: zynkOpTokenAccount, // SAME as zow - self-
transfer!
        orderTracker: orderTrackerPDA,
        systemProgram: SystemProgram.programId,
        tokenProgram: TOKEN_PROGRAM_ID,
        sysvarInstructions: null,
      })
      .signers([manager, zynkOpWallet])
      .rpc();

    const zowBalanceAfter = await getAccount(provider.connection,
zynkOpTokenAccount);
    const tracker = await program.account.orderTracker.fetch(orderTrackerPDA);

    assert.equal(
      zowBalanceBefore.amount.toString(),
      zowBalanceAfter.amount.toString(),
      "Balance should be unchanged (self-transfer is no-op)"
    );
    assert.ok(tracker.amountOut.toNumber() === 500_000_000, "Phantom debt of
500M recorded");
  });
```

Zynk Team's Comment

Extended the check to exclude any ZOW token accounts as beneficiary.

Fixed in Commit hash

38a2c66d2d12d047a136b46af729c75258e375b4

Delimiter Injection in attest_order Signature Messages

Resolved

Path

lib.rs

Description

The attest_order signed message uses :: delimiters with unvalidated string inputs (origin, proxy_asset, target_chain, proxy_txn). If any input contains ::, different parameter combinations produce identical message bytes, enabling signature reuse across different routing paths. This is a structural cryptographic issue distinct from V1 L-6 (which covers general input length/format validation).

```
// lib.rs:640 - Delimiter injection via free-form string inputs
let message = format!("{}::{}::{}::{}::{}", DOMAIN_SEPARATOR, origin,
proxy_asset, target_chain, proxy_txn);

// Example collision:
// proxy_asset = "0xBridge::relay", target_chain = "0xRecv"
// produces IDENTICAL bytes as:
// proxy_asset = "0xBridge", target_chain = "relay::0xRecv"
```

Remediation Suggestion

Use length-prefixed serialization (e.g., Borsh) or validate inputs don't contain :::

```
require!(!origin.contains("::") && !proxy_asset.contains("::") &&
         !target_chain.contains("::") && !proxy_txn.contains("::"),
         CustomError::InvalidInput);
```

POC

```
it("PoC-9: Delimiter injection creates signature collision (F-03)", async () =>
{
    const origin = "eth";
    const proxy1 = "0xBridge::relay";
    const target1 = "0xRecv";
    const txn1 = "0xabc";

    const proxy2 = "0xBridge";
    const target2 = "relay::0xRecv";
    const txn2 = "0xabc";

    const msg1 = `${DOMAIN_SEPARATOR}::${origin}::${proxy1}::${target1}::$
{txn1}`;
    const msg2 = `${DOMAIN_SEPARATOR}::${origin}::${proxy2}::${target2}::$
{txn2}`;

    assert.equal(msg1, msg2, "Messages should be identical despite different
parameters");
    const { signature: sig1 } = buildEd25519Ix(msg1, manager);
    const { signature: sig2 } = buildEd25519Ix(msg2, manager);
    assert.deepEqual(Array.from(sig1), Array.from(sig2), "Signatures should be
identical");
  });
```

Fixed in Commit hash

fa321221bde423e3e96db2e02d332017c5730984

Zero-Amount Transfer in pull_and_create_order

Resolved

Path

lib.rs

Description

Unlike create_order which guards with if amount != 0 before transferring (line 481), pull_and_create_order has no zero-amount guard. With amount=0, two zero-amount SPL transfers proceed (PDV->ZOW, ZOW->beneficiary), an OrderTracker is created with amount_in=0, amount_out=0, and the order is immediately closeable since the deficiency check passes (0 >= 0). This creates costless orders that pollute on-chain state and emit misleading events.

```
// lib.rs:346-430 - pull_and_create_order has no zero-amount check
pub fn pull_and_create_order(…, amount: u64, …) -> Result<()> {
    // No `require!(amount > 0)` check here
    // Both SPL transfers proceed with amount=0 (no-ops)
    // OrderTracker created with amount_in=0, amount_out=0
}

// Compare with create_order (lib.rs:481) which HAS a guard:
if amount != 0 {
    // … token transfer only happens for non-zero amounts
}
```

Remediation Suggestion

```
// Add at the start of pull_and_create_order:
require!(amount > 0, CustomError::InvalidAmount);
```

POC

```
it("PoC-4: Zero-amount pull_and_create_order succeeds (F-17)", async () => {
    const orderId = generateOrderId();
    const beneficiary = beneficiaryWallet.publicKey;
    const [orderTrackerPDA] = PublicKey.findProgramAddressSync(
      [Buffer.from("order_tracker"), beneficiary.toBuffer(), orderId],
      program.programId
    );

    await program.methods
      .pullAndCreateOrder(Array.from(partnerId), Array.from(orderId), new
anchor.BN(0), null, null)
      .accounts({
        config: configPDA,
        manager: manager.publicKey,
        partnerDepositVault: partnerDepositVaultPDA,
        pdvTokenAccount: partnerDepositTokenAccount,
        zynkOpWallet: zynkOpWallet.publicKey,
        zowTokenAccount: zynkOpTokenAccount,
        beneficiaryTokenAccount: beneficiaryTokenAccount,
        orderTracker: orderTrackerPDA,
        systemProgram: SystemProgram.programId,
        tokenProgram: TOKEN_PROGRAM_ID,
        sysvarInstructions: null,
      })
      .signers([manager, zynkOpWallet])
      .rpc();
```

```
    const tracker = await program.account.orderTracker.fetch(orderTrackerPDA);
    assert.ok(tracker.amountIn.toNumber() === 0, "amount_in should be 0");
    assert.ok(tracker.amountOut.toNumber() === 0, "amount_out should be 0");
  });
```

Fixed in Commit hash

ac18663cf2da057c821eafcc4c2d3fcfd1f0a0d0

Incomplete Ed25519 Signature Verification — Missing num_signatures and Offset Validation

Resolved

Path

lib.rs

Description

The verify_signature_syscall function checks raw byte data at hardcoded positions within the Ed25519 instruction but never validates the num_signatures header field or the offset/index fields that tell the Ed25519 precompile where to read the pubkey, signature, and message. An Ed25519 instruction can be constructed that passes all of the program's checks while the precompile actually verifies against a completely different public key.

```
pub fn verify_signature_syscall(
    ix_sysvar_account: &AccountInfo,
    signer_pubkey: &Pubkey,
    msg: String,
    signature: [u8; 64]
) -> Result<()> {
    let ed25519_instruction_result = load_instruction_at_checked(0,
ix_sysvar_account);
    // …
    let data = &ed25519_instruction.data;
    let message: Vec<u8> = msg.into_bytes();

    // ❌ Only checks program_id, account count, and total length
    // ❌ Never checks data[0] (num_signatures)
    // ❌ Never checks bytes 2-15 (offset/index fields)
    if ed25519_instruction.program_id != ED25519_ID
        || ed25519_instruction.accounts.len() != 0
        || data.len() != 16 + 32 + 64 + message.len() {
        return Err(ProgramError::InvalidInstructionData.into());
    }

    // Reads data at FIXED positions, assumes offsets point here
    let data_pubkey = &data[16..48];
    let data_signature = &data[48..112];
    let data_message = &data[112..];
    if data_pubkey != &signer_pubkey.to_bytes()
        || data_signature != signature
        || data_message != message {
        return Err(CustomError::InvalidEd25519Message.into());
    }
    Ok(())
}
```

POC

Bytes	Field
0	`num_signatures` (u8)
1	padding
2–3	`signature_offset` (u16)
4–5	`signature_instruction_index` (u16)
6–7	`public_key_offset` (u16)
8–9	`public_key_instruction_index` (u16)
10–11	`message_data_offset` (u16)
12–13	`message_data_size` (u16)
14–15	`message_instruction_index` (u16)
16–47	public key (32 bytes)
48–111	signature (64 bytes)
112+	message

Attack via offset manipulation (num_signatures = 1):

- Attacker places the manager's pubkey at data[16..48] (to pass the code's check).
  & Attacker sets public_key_instruction_index = 1 (pointing to the program instruction) and public_key_offset to an offset within that instruction's data where the attacker's pubkey resides.
- Attacker signs the message with their own private key, places this valid signature at data[48..112].
- The Ed25519 precompile reads the attacker's pubkey from instruction 1 and verifies successfully.
- The program's checks pass: data[16..48] == manager_pubkey ✓, data[48..112] == signature ✓, data[112..] == message ✓.
- Signature verification is "passed" but was never verified against the manager's key.

Attack via num_signatures = 0 (on runtimes without ed25519_precompile_verify_strict):

- Set data[0] = 0. The Ed25519 precompile returns Ok(()) without verifying anything.
- Fill data[16..] with the expected pubkey, signature, and message.
- The program's checks all pass. No cryptographic verification was performed.

Recommendation

```
// Verify num_signatures == 1
require!(data[0] == 1, ProgramError::InvalidInstructionData);


// Verify offsets point to expected positions
let sig_offset = u16::from_le_bytes([data[2], data[3]]);
let sig_ix_idx = u16::from_le_bytes([data[4], data[5]]);
let pk_offset  = u16::from_le_bytes([data[6], data[7]]);
let pk_ix_idx  = u16::from_le_bytes([data[8], data[9]]);
let msg_offset = u16::from_le_bytes([data[10], data[11]]);
let msg_size   = u16::from_le_bytes([data[12], data[13]]);
let msg_ix_idx = u16::from_le_bytes([data[14], data[15]]);


require!(
    pk_offset == 16 && pk_ix_idx == 0xFFFF &&
    sig_offset == 48 && sig_ix_idx == 0xFFFF &&
    msg_offset == 112 && msg_ix_idx == 0xFFFF &&
    msg_size == message.len() as u16,
    ProgramError::InvalidInstructionData
);
```

Fixed in Commit hash

d5eeee0271376ba9f8c7b227b17f7f22d77e24f4
Add this too: 5bb6d021cca703679b12fffd94f9f88581c21686

Zynk Team's Comment

As, we have been preaching continuously since the beginning - there are no external entities involved in our protocol. The Ed25519 message building and signing happens within our system by the Manager (after verification of the records stored). The transaction containing Ed25519 signature is sent again by our system, all the time requiring the signature of the Manager and few others (protocol's signers - trusted) when required in the context.

Low Severity Issues

Arithmetic Operations Should Use Checked Math

Resolved

Path

lib.rs

Description

While the Cargo.toml correctly enables overflow-checks = true, the code still uses raw arithmetic operators in places:

```
order_tracker.amount_in += amount;
```

The workspace Cargo.toml correctly enables overflow protection:

```
[profile.release]
overflow-checks = true
```

This means arithmetic overflow will cause a program panic rather than silent wrapping. While the protocol is protected from wrap-around exploits, the panic behavior provides poor user experience and no custom error messaging.

Impact

- If overflow occurs, the transaction fails with a generic panic rather than a descriptive CustomError Difficult to diagnose in production
- Panics are less informative than explicit error handling
- Violates Solana best practices for explicit arithmetic safety

Recommendation

Use checked_add for explicit, descriptive error handling:

```
order_tracker.amount_in = order_tracker.amount_in
    .checked_add(amount)
    .ok_or(CustomError::ArithmeticOverflow)?;
```

This provides consistent error handling, clear error messages for debugging, and defense-in-depth regardless of build profile settings.

Fixed In

b25988df09a5e77ef92778286d6c982a33887ccc

No Duplicate Check in Whitelisted Token Mints

Resolved

Path

lib.rs

Description

The initialize function (L311-L315) validates that whitelisted mints are non-empty and not the null address, but does not check for duplicates:

```
require!(whitelisted_token_mints.len() > 0,
CustomError::EmptyWhitelistedTokenMints);
for token_mint in whitelisted_token_mints.iter() {
    validate_address(token_mint)?;
}
config.whitelisted_token_mints = whitelisted_token_mints;
```

Impact

- Duplicate entries waste account space (each Pubkey is 32 bytes).
- The Config account's space is pre-calculated from the vector length, so duplicates increase rent cost unnecessarily.
- config.whitelisted_token_mints.contains() iterates linearly — duplicates increase gas cost for every order.

Recommendation

Add deduplication logic:

```
let mut sorted = whitelisted_token_mints.clone();
sorted.sort();
sorted.dedup();
require!(sorted.len() == whitelisted_token_mints.len(),
CustomError::DuplicateTokenMint);
```

Fixed in Commit hash

f2cfde07e057c3270cbce4433621f4911310eb93

No Support for Token-2022

Resolved

Path

lib.rs

Description

The codebase explicitly uses anchor_spl::token (L2) and token::transfer (L395, L404, L489, L569). These are specific to the legacy SPL Token Program. If the protocol attempts to whitelist or interact with a Token-2022 mint (e.g., new stablecoins like PYUSD or advanced assets with transfer hooks), the CPI calls will fail because the program ID will mismatch or the interface will not be respected.

Impact

The protocol is currently incompatible with any Token-2022 asset.

Recommendation

If support for Token-2022 is desired, refactor the code to use anchor_spl::token_interface and utilize transfer_checked instead of transfer, which dynamically dispatches to the correct program based on the mint's owner.

Fixed in

76a31efba80c4be15c562324554b0e60d1d72ef6

Attest_order signature does not include amount

Resolved

Path

lib.rs

Description

The attest_order function verifies an Ed25519 signature from the manager. The signed message includes origin, proxy, target, and txn. Crucially, the amount parameter is NOT included in the signed message. While the Manager is currently a Signer (preventing external replay), amount is an integral part of the order and should be included in the signed message.

Recommendation

Include amount in the signed message payload immediately to future-proof the protocol.

Fixed in

5f49c8833603aaf28c236ba9b707e40b0312c0ab

Custom close_account Does Not Zero Discriminator — Account Revival Attack

Resolved

Path

lib.rs

Description

The program uses a custom close_account helper function (L265-L274) to close accounts:

```
pub fn close_account<'a, 'b>(from: impl ToAccountInfo<'a>, to: impl
ToAccountInfo<'b>) -> Result<()> {
    let from = from.to_account_info();
    let to = to.to_account_info();
    **to.try_borrow_mut_lamports()? += **from.try_borrow_lamports()?;
    **from.try_borrow_mut_lamports()? = 0;
    from.try_borrow_mut_data()?.fill(0);
    Ok(())
}
```

While it zeroes the data, the Solana runtime does not mark this account as “closed” within the same transaction. Between the close_account call and the end of the transaction, the Solana runtime may re-credit the account with lamports (if other instructions in the same transaction reference it), or another program can send lamports to it.

Critically, this function is used to close OrderTracker accounts in several places:

`pull_and_create_order (L408)` — transient orders
`create_order (L494)` — transient orders
`replenish (L582)` — order closure
`attest_order (L661)` — attest closure
`execute_consensus (L1004)` — timelock closure

If an attacker includes an instruction later in the same transaction that sends lamports back to the same account, the account is “revived” with zeroed data. Since attest_order uses init_if_needed, Anchor will see the account exists (has lamports) but has zeroed data , and reinitialize it, potentially allowing replay of attestations.

This is different from Anchor’s built-in close constraint, which transfers lamports to the destination AND assigns the account’s owner to the System Program, preventing deserialization by the program in subsequent instructions.

Impact

Account revival attacks are possible within the same transaction.
Combined with init_if_needed in attest_order, this enables reinitialization of order trackers.
Potential for double-attestation or double-spend scenarios.

Recommendation

Replace the custom close_account with Anchor’s built-in close constraint wherever possible. If manual closure is needed, also assign the account owner to the System Program:

```
pub fn close_account<'a, 'b>(from: impl ToAccountInfo<'a>, to: impl
ToAccountInfo<'b>) -> Result<()> {
    let from = from.to_account_info();
    let to = to.to_account_info();
    **to.try_borrow_mut_lamports()? += **from.try_borrow_lamports()?;
    **from.try_borrow_mut_lamports()? = 0;
    from.try_borrow_mut_data()?.fill(0);
    from.assign(&anchor_lang::solana_program::system_program::ID); // Prevent
revival
    Ok(())
}
```

Reference

https://rareskills.io/post/solana-close-account

Zynk Team's Comment

Order closures are performed in a controlled environment, plus, it doesn't directly affect the funds or operational continuity of the protocol

Fixed in Commit hash

b34c31fb35503d17f65097da742296910b667a68

Informational Issues

Monolithic Code Structure

Acknowledged

Path

lib.rs

Description

The entire protocol implementation (~985 lines) resides in a single lib.rs file. This includes instruction handlers, account structs, context definitions, error enums, events, constants, and utility functions all mixed together.

A well-structured Anchor project typically organizes code into separate modules:

```
programs/zynk-protocol/src/
├── lib.rs              # Program entrypoint and instruction dispatch
├── instructions/       # One file per instruction
│   ├── initialize.rs
│   ├── create_order.rs
│   ├── replenish.rs
│   ├── close_order.rs
│   └── ...
├── state/              # Account structs
│   ├── config.rs
│   ├── order_tracker.rs
│   └── request.rs
├── error.rs            # Custom error definitions
├── events.rs           # Event structs
└── constants.rs        # Seeds, sizes, etc.
```

Impact

- **Maintainability:** Large single files are harder to navigate, review, and modify
- **Review Difficulty:** Auditors and developers must scroll through unrelated code
- **Merge Conflicts:** Multiple developers editing the same file increases conflict risk
- **Testing:** Harder to unit test individual components in isolation

Recommendation

Refactor into the standard Anchor project structure. This is non-functional but improves long-term maintainability and makes future audits more efficient.

Zynk Team's Comment

Modularization is in our pipelines later.

execute_consensus Double-Close on Timelock Account

Resolved

Path

lib.rs

Description

The execute_consensus function (L974-L1007) explicitly calls close_account(req, &ctx.accounts.guardian) at the end of the handler. However, the Ack context struct for this function does not use close = guardian on the timelock account — so the manual close_account call handles the closure.

This is not a vulnerability in itself, but the manual close_account implementation (which zeroes data and moves lamports) duplicates what Anchor's close constraint does with additional safety guarantees. If a future developer adds close = guardian to the Ack struct (for consistency), it would result in a double-close attempt.

Recommendation

Use Anchor's native close constraint on the Ack struct's timelock field instead of the manual close_account call, or add a clear comment explaining why manual closure is necessary here.

Zynk Team's Comment

The Ack context struct is being used for a different method too, wherein the timelock account must not be closed. Adding comment to prevent confusion.

Fixed in Commit hash

94de5f24442162a210adfc199ee07b380dde192a

Manual Struct Size Calculation

Resolved

Path

lib.rs

Description

The codebase manually calculates account space in several places, for example in Config::space:

```
pub fn space(n: u32) -> usize {
    return 8 + 32 + 32 + 32 + 32 + 4 + (32 * n as usize) + 1;
}
```

Manually summing byte sizes is error-prone and hard to maintain. If a struct field is added or changed, the manual calculation must be updated, or the program may fail at runtime or waste rent.

Recommendation

Use Anchor's #[derive(InitSpace)] macro to automatically calculate the space required for the account. This ensures the space calculation is always in sync with the struct definition.

```
#[account]
#[derive(InitSpace)]
pub struct Config {
    // fields...
}

// In context:
space = 8 + Config::INIT_SPACE,
```

Fixed in

3dee3997667853c5ce75b545f61bdad7b80d6e04

Closing Summary

In this report, we have considered the security of Zynk Labs Contract. We performed our audit according to the procedure described above.

1 issue of High severity, 7 issues of Medium severity, 6 issues of Low severity & 3 issues of informational severity were found. One issue was acknowledged by the Zynk Labs Team and the other issues were resolved.

Disclaimer

At QuillAudits, we have spent years helping projects strengthen their smart contract security. However, security is not a one-time event—threats evolve, and so do attack vectors. Our audit provides a security assessment based on the best industry practices at the time of review, identifying known vulnerabilities in the received smart contract source code.

This report does not serve as a security guarantee, investment advice, or an endorsement of any platform. It reflects our findings based on the provided code at the time of analysis and may no longer be relevant after any modifications. The presence of an audit does not imply that the contract is free of vulnerabilities or fully secure.

While we have conducted a thorough review, security is an ongoing process. We strongly recommend multiple independent audits, continuous monitoring, and a public bug bounty program to enhance resilience against emerging threats.

Stay proactive. Stay secure.

About QuillAudits

QuillAudits is a leading name in Web3 security, offering top-notch solutions to safeguard projects across DeFi, GameFi, NFT gaming, and all blockchain layers. With eight years of expertise, we've secured over 1500 projects globally, averting over $3 billion in losses. Our specialists rigorously audit smart contracts and ensure DApp safety on major platforms like Ethereum, BSC, Arbitrum, Algorand, Tron, Polygon, Polkadot, Fantom, NEAR, Solana, and others, guaranteeing your project's security with cutting-edge practices.

8+ Years of Expertise	**1M+** Lines of Code Audited
50+ Chains Supported	**1500+** Projects Secured

Follow Our Journey

AUDIT REPORT

February 2026

For

Zynk

QuillAudits

Canada, India, Singapore, UAE, UK

www.quillaudits.com audits@quillaudits.com